SQL DATA ANALYSIS PROJECT



PIZZA SALES ANALYSIS
Using SQL

A Data-Driven Exploration of Pizza Sales Analyzing sales performance, customer preferences and revenue trends using SQL queries.

PRESENTED BY: Anitha N

SQL PROJECT | 2026

PROJECT INTRODUCTION

Understanding pizza sales through SQL-based data analysis

01 | Project Objective

To analyze pizza sales data using SQL and extract meaningful insights about orders, revenue, pizza preferences, and sales performance.

02 | Project Goals

Retrieve total orders and revenue.
Identify popular pizza sizes and types.
Analyze category-wise sales.
Understand hourly and daily order patterns.
Identify top revenue-generating pizzas.

03 | Tools & Technologies

SQL
Query language

MySQL
Database



DATABASE OVERVIEW

Pizza Sales Database – Four Tables

1. **ORDERS**
   - order_id (Primary Key)
   - date
   - time
2. **ORDER_DETAILS**
   - order_details_id (Primary Key)
   - order_id (Foreign Key)
   - pizza_id (Foreign Key)
   - quantity
3. **PIZZAS**
   - pizza_id (Primary Key)
   - pizza_type_id (Foreign Key)
   - size
   - price
4. **PIZZA_TYPES**
   - pizza_type_id (Primary Key)
   - name
   - category
   - ingredients

DATABASE RELATIONSHIPS

orders → order_details (order_id)

order_details → pizzas (pizza_id)

pizzas → pizza_types (pizza_type_id)

KEY HIGHLIGHTS

4 relational database tables

Primary Keys (PK) identify unique records.

Foreign Keys (FK) establish relationships between tables.

JOIN operations combine data for sales analysis.

Retrieve the total number of orders placed

```sql
SELECT
    COUNT(order_id) AS total_orders
FROM
    orders;
```

Result Grid

	total_orders
▶	21350

Calculate the total revenue generated from pizza sales

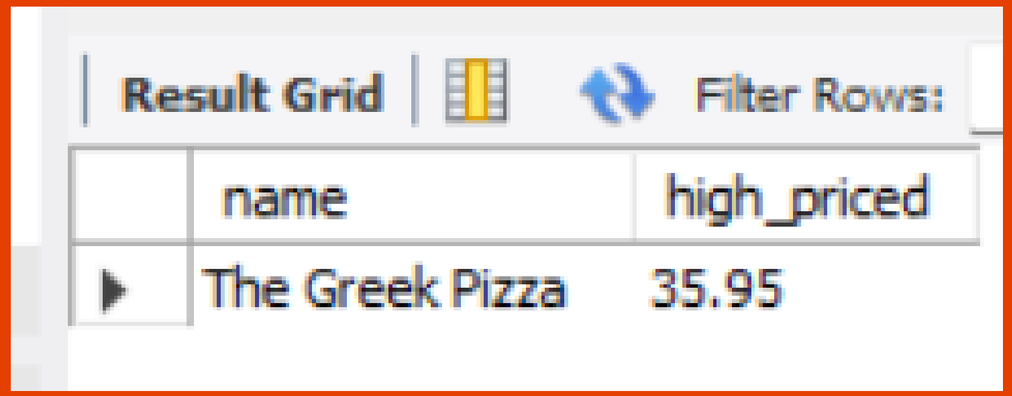
```sql
SELECT
    ROUND(SUM(o.quantity * p.price), 2) AS toatal_revenue
FROM
    order_details o
        JOIN
    pizzas p ON o.pizza_id = p.pizza_id;
```

Result Grid

	toatal_revenue
▶	817860.05

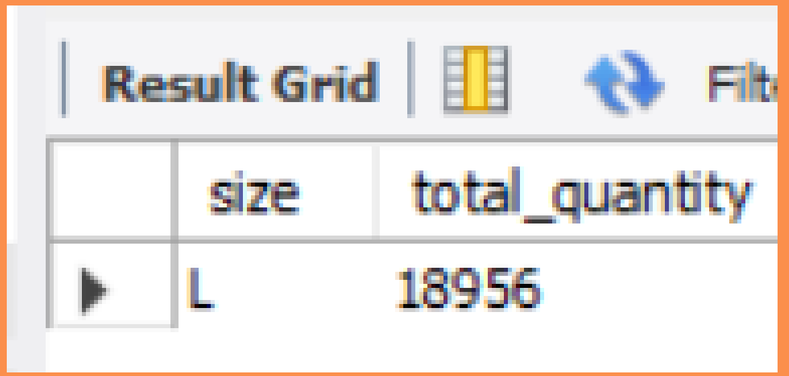
Identify the highest-priced pizza

```sql
SELECT
    pt.name, p.price AS high_priced
FROM
    pizza_types pt
        JOIN
    pizzas p ON pt.pizza_type_id = p.pizza_type_id
ORDER BY high_priced DESC
LIMIT 1;
```



Identify the most common pizza size ordered

```sql
SELECT
    p.size, SUM(od.quantity) AS total_quantity
FROM
    pizzas p
        JOIN
    order_details od ON p.pizza_id = od.pizza_id
GROUP BY p.size
ORDER BY total_quantity DESC
LIMIT 1;
```



List the top 5 most ordered pizza types along with their quantities

```sql
SELECT
    pt.name, SUM(od.quantity) AS quantity
FROM
    pizza_types pt
        JOIN
    pizzas p ON pt.pizza_type_id = p.pizza_type_id
        JOIN
    order_details od ON p.pizza_id = od.pizza_id
GROUP BY pt.name
ORDER BY quantity DESC
LIMIT 5;
```

Result Grid | Filter Rows:

name	quantity
The Classic Deluxe Pizza	2453
The Barbecue Chicken Pizza	2432
The Hawaiian Pizza	2422
The Pepperoni Pizza	2418
The Thai Chicken Pizza	2371

Join the necessary tables to find the total quantity of each pizza category ordered

```sql
SELECT
    pt.category, SUM(od.quantity) AS quantity
FROM
    pizza_types pt
        JOIN
    pizzas p ON pt.pizza_type_id = p.pizza_type_id
        JOIN
    order_details od ON p.pizza_id = od.pizza_id
GROUP BY pt.category
ORDER BY quantity DESC;
```

Result Grid

category	quantity
Classic	14888
Supreme	11987
Veggie	11649
Chicken	11050

Determine the distribution of orders by hour of the day

```sql
SELECT
    HOUR(order_time) AS order_hour, COUNT(order_id) AS order_count
FROM
    orders
GROUP BY order_hour;
```

Result Grid | Filter Ro

order_hour	order_count
11	1231
12	2520
13	2455
14	1472
15	1468
16	1920
17	2336
18	2399
19	2009
20	1642
21	1198
22	663
23	28
10	8
9	1

Join relevant tables to find the category-wise distribution of pizzas

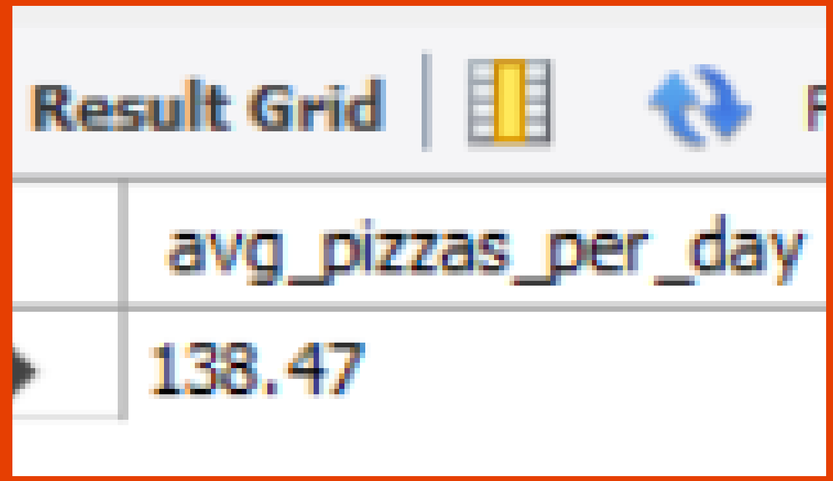
```sql
SELECT
    pt.category,
    COUNT(od.order_details_id) AS total_pizza_records
FROM
    pizza_types pt
        JOIN
    pizzas p ON pt.pizza_type_id = p.pizza_type_id
        JOIN
    order_details od ON p.pizza_id = od.pizza_id
GROUP BY pt.category;
```

Result Grid | Filter Rows:

category	total_pizza_records
Classic	14579
Veggie	11449
Supreme	11777
Chicken	10815

Group the orders by date and calculate the average number of pizzas ordered per day

```sql
SELECT
    ROUND(AVG(daily_quantity), 2) AS avg_pizzas_per_day
FROM
    (SELECT
        o.order_date, SUM(od.quantity) AS daily_quantity
    FROM
        orders o
    JOIN order_details od ON o.order_id = od.order_id
    GROUP BY o.order_date) AS daily_orders;
```



Determine the top 3 most ordered pizza types based on revenue

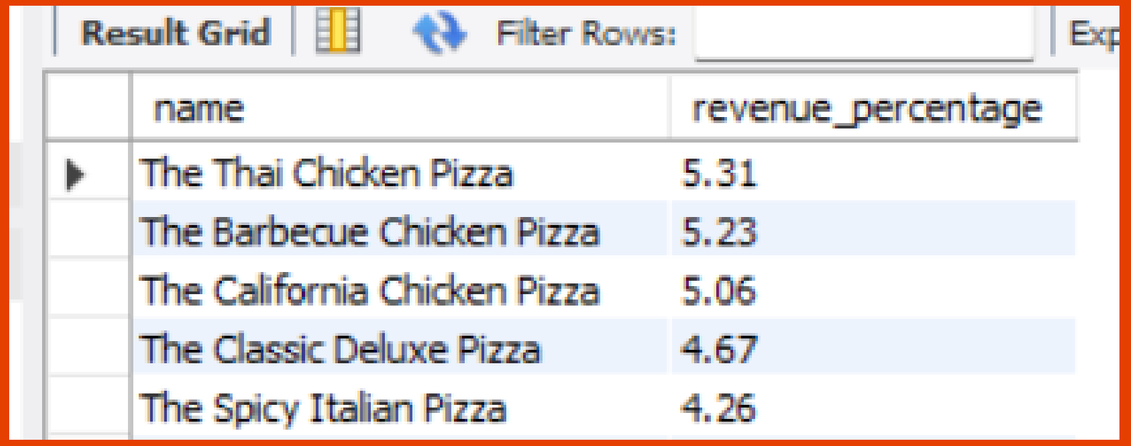
```sql
SELECT
    pt.name, SUM(od.quantity * p.price) AS revenue
FROM
    pizza_types pt
        JOIN
    pizzas p ON pt.pizza_type_id = p.pizza_type_id
        JOIN
    order_details od ON p.pizza_id = od.pizza_id
GROUP BY pt.name
ORDER BY revenue DESC
LIMIT 3;
```

Result Grid | Filter Rows:

name	revenue
The Thai Chicken Pizza	43434.25
The Barbecue Chicken Pizza	42768
The California Chicken Pizza	41409.5

Calculate the percentage contribution of each pizza type to total revenue

```sql
select pt.name,
round(sum(od.quantity* p.price)*100/sum(sum(od.quantity*p.price)) over(), 2) as revenue_percentage
from pizza_types pt
join pizzas p
on pt.pizza_type_id = p.pizza_type_id
join order_details od
on p.pizza_id = od.pizza_id
group by pt.name
order by revenue_percentage desc;
```



name	revenue_percentage
The Thai Chicken Pizza	5.31
The Barbecue Chicken Pizza	5.23
The California Chicken Pizza	5.06
The Classic Deluxe Pizza	4.67
The Spicy Italian Pizza	4.26

Analyze the cumulative revenue generated over time

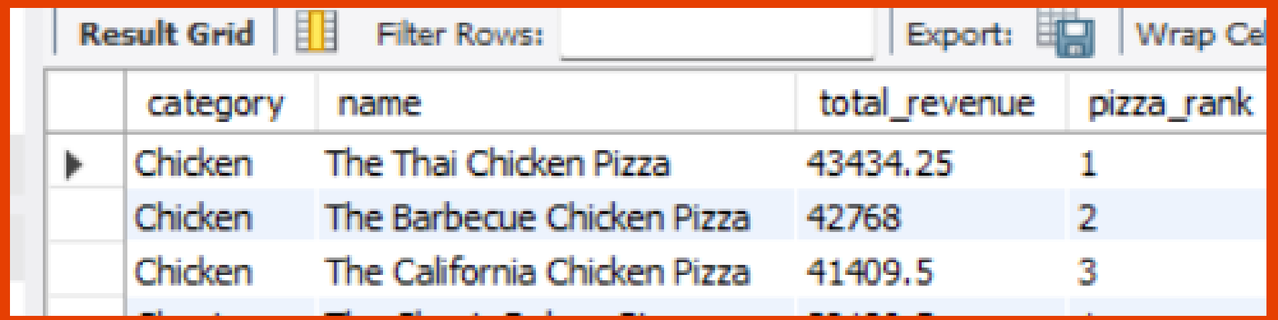
```sql
select o.order_date,
round(sum(sum(od.quantity*p.price)) over(order by o.order_date),2) as cumulative_revenue
from pizzas p
join order_details od
on p.pizza_id = od.pizza_id
join orders o
on od.order_id = o.order_id
group by o.order_date
order by o.order_date;
```

Result Grid | Filter Rows:

order_date	cumulative_revenue
2015-01-01	2713.85
2015-01-02	5445.75
2015-01-03	8108.15
2015-01-04	9863.6

Determine the top 3 most ordered pizza types based on revenue for each pizza category

```sql
with pizza_revenue as (select pt.category,pt.name, round(sum(od.quantity*p.price),2) as total_revenue
from pizza_types pt
join pizzas p on pt.pizza_type_id = p.pizza_type_id
join order_details od on p.pizza_id = od.pizza_id
group by pt.category, pt.name),
ranked_pizzas as
(select category, name, total_revenue,
dense_rank() over (partition by category order by total_revenue desc) as pizza_rank
from pizza_revenue)
select category, name, total_revenue, pizza_rank
from ranked_pizzas
where pizza_rank <=3
order by category, pizza_rank;
```



category	name	total_revenue	pizza_rank
Chicken	The Thai Chicken Pizza	43434.25	1
Chicken	The Barbecue Chicken Pizza	42768	2
Chicken	The California Chicken Pizza	41409.5	3